

上海市中小学生人工智能算法设计活动

C++ 复赛初中组

时间：2025 年 12 月 28 日 13:00 ~ 16:30

题目名称	座位	拼数	多边形	异或和
题目类型	传统题	传统题	传统题	传统题
目录	seat	number	polygon	xor
可执行文件名	seat	number	polygon	xor
输入文件名	seat.in	number.in	polygon.in	xor.in
输出文件名	seat.out	number.out	polygon.out	xor.out
每个测试点时限	2.0 秒	2.0 秒	1.0 秒	1.0 秒
内存限制	512 MiB	512 MiB	512 MiB	512 MiB
测试点/子任务数	20	20	25	8
测试点是否等分	是	是	是	否

提交源程序文件名

对于 C++ 语言	seat.cpp	number.cpp	polygon.cpp	xor.cpp
-----------	----------	------------	-------------	---------

编译选项

对于 C++ 语言	-O2 -std=c++14 -static
-----------	------------------------

注意事项（请仔细阅读）

1. 文件名（程序名和输入输出文件名）必须使用英文小写。
2. `main()` 函数的返回值类型必须是 `int`，程序正常结束时的返回值必须是 0。
3. 若无特殊说明，结果的比较方式为全文比较（过滤行末空格及文末换行）。
4. 选手提交的程序源文件大小不得超过 100KiB。
5. 提交的程序代码文件需要放在与题目英语名称相同的子文件夹中。
6. 程序可使用的栈空间内存限制与题目的内存限制一致。
7. 禁止在源代码中改变编译器参数（如使用 `#pragma` 命令），禁止使用系统结构相关指令（如内联汇编）或其他可能造成不公平的方法。
8. 因违反上述规定而出现的问题，申诉时一律不予受理。
9. 只提供 Linux 格式附加样例文件。
10. 统一评测时采用的机器配置为：Intel Core Ultra 9 285K CPU @ 3.70 GHz（关闭睿频与能效核），内存 32 GB。上述时限以此配置为准。
11. 评测在当前最新公布的 NOI Linux 下进行，各语言的编译器版本以此为准。

座位 (seat)

【题目描述】

上海市中小学生人工智能算法设计活动的复赛正在进行。小 R 所在的考点共有 10^9 名考生，其中所有考生的初赛成绩互不相同。所有 10^9 名考生将按照初赛第一轮的成绩，由高到低抽签分配考场。

具体地，考场共有 n 个，且形成一棵有根树。1 号考场为根结点。每个考生可自行选择开始抽签的考场。若他当前位于考场 u ，则他从 u 和 u 的所有儿子考场中等概率随机抽取一个。若他抽中考场 u ，则他在考场 u 考试，不再继续抽签；否则，他会来到他抽中的儿子考场 v ，并在考点 v 重复上述抽签过程，直到确定在某个考场考试为止。

考试组织者小 D 想让你动态地计算每个考场的期望人数，以便给每个考场安排适量的座位。下面将会有 q 个事件，每个事件形如：一位考生选择 u 号考场开始抽签，从而确定自己考试的考场（抽签过程不公开）；或者小 D 询问你考场 u 的期望人数。只需求出答案在 mod 998244353 意义下的值即可。

对期望和取模的定义和计算方法，请看提示。

【输入格式】

从文件 *seat.in* 中读入数据。

第一行，一个正整数 c ，表示测试点编号。对于样例， $c = 0$ 。

第一行，共两个正整数 n, q 。

接下来 $n - 1$ 行，每行两个正整数 u, v ，表示考场树中的一条边。

接下来 q 行，每行两个正整数 op, u 。若 $op = 1$ ，则下一位考生在 u 号考场开始抽签，并完成抽签过程。若 $op = 2$ ，则查询目前 u 号考场的期望人数。

【输出格式】

输出到文件 *seat.out* 中。

对于每个 $op = 2$ 的操作，输出一行一个整数，表示询问的答案。

【样例 1 输入】

```
1 0
2 3 5
3 1 2
4 2 3
5 1 1
6 1 3
```

7	2 1
8	2 2
9	2 3

【样例 1 输出】

1	499122177
2	748683265
3	748683266

【样例 1 解释】

第一位考生在 1 号考场抽签，有 $\frac{1}{2}$ 的概率直接在 1 号考场考试，剩下 $\frac{1}{2}$ 的概率在 2 号考场继续抽签，有 $\frac{1}{4}$ 的概率在 2 号考场考试，有 $\frac{1}{4}$ 的概率在 3 号考场考试。

第二位考生只能在 3 号考场考试。

取模前答案分别为 $\frac{1}{2}, \frac{1}{4}, \frac{5}{4}$ 。

【样例 2】

见选手目录下的 *seat/seat2.in* 与 *seat/seat2.ans*。

该样例满足测试点 1 ~ 2 的约束条件。

【样例 3】

见选手目录下的 *seat/seat3.in* 与 *seat/seat3.ans*。

该样例满足测试点 3 ~ 6 的约束条件。

【样例 4】

见选手目录下的 *seat/seat4.in* 与 *seat/seat4.ans*。

该样例满足测试点 7 ~ 10 的约束条件。

【样例 5】

见选手目录下的 *seat/seat5.in* 与 *seat/seat5.ans*。

该样例满足测试点 11 ~ 14 的约束条件。

【样例 6】

见选手目录下的 *seat/seat6.in* 与 *seat/seat6.ans*。

该样例满足测试点 15 ~ 18 的约束条件。

【样例 7】

见选手目录下的 *seat/seat7.in* 与 *seat/seat7.ans*。

该样例满足测试点 19 ~ 20 的约束条件。

【数据范围】

测试点编号	$n, q \leq$	特殊性质
1 ~ 2	10	C
3 ~ 6	10^3	C
7 ~ 10	5×10^5	AC
11 ~ 14	5×10^5	BC
15 ~ 18	5×10^5	C
19 ~ 20	5×10^5	无

特殊性质 A：除 1 号点以外，所有点的父亲是 1 号点。

特殊性质 B：所有 $op = 1$ 的操作中都有 $u = 1$ 。

特殊性质 C：所有 $op = 1$ 在 $op = 2$ 之前。

【提示】

对期望的定义：一个取值有限的随机整数 x ，它的期望等于 x 在所有概率下的平均值。具体地，设 x 的所有可能值为 $x_1, x_2, \dots, x_k$ ，取这些值的概率分别为 $y_1, y_2, \dots, y_k$ ，则 x 的期望为 $x_1y_1 + x_2y_2 + \dots + x_ky_k$ 。本题中，考场 u 人数的期望也可以计算为：每个人在考场 u 考试的概率之和。

本题需要对有理数进行取模。具体方法如下：

对某个素数 p 和每个 $\text{mod } p$ 不为 0 的整数 q ，可以找到唯一的整数 $r \in [1, p-1]$ 使得 $qr \equiv 1 \pmod{p}$ 。这时称 r 是 q 在 $\text{mod } p$ 意义下的乘法逆元，记作 $r \equiv q^{-1} \pmod{p}$ 。当你需要将一个数除以 q 时，你只需要乘以 q^{-1} 即可。可以证明， q^{-1} 与 q^{p-2} 在 $\text{mod } p$ 意义下同余，可以用快速幂求解 q^{-1} 。

拼数 (number)

【题目描述】

小 R 正在学习多进制数处理。小 X 给小 R 一个数位集合 $S = \{1, 2, \dots, n\}$, 让小 R 用 S 中的数位拼成一些 $n+1$ 进制数。

小 R 认为, 一些数位构成的数位对 (x, y) 是优美的。(不区分 x 和 y 的顺序, 且保证 $x \neq y$) 他希望对于拼出的数, 数位两两之间有很多优美的数位对。所以, 他决定只使用 S 的一个子集 T 来拼数, 且希望 T 中每个数位至少与 k 个其它 T 中的数位构成优美的数位对。

小 R 希望你能帮他求出最大的 k 使得他能找到 T , 并在 k 取此最大值的情况下找到最大的满足上述条件的 T 。可以证明最大的 T 是唯一的。

【输入格式】

从文件 `number.in` 中读入数据。

第一行, 一个正整数 n 。

后面 n 行, 第 i 行先读入一个正整数 k_i , 表示 i 与大于 i 的数位构成的优美数位对的数量。再读入 k_i 个不同的正整数 $u_{i,j} > i$, 表示 $(i, u_{i,j})$ 是优美的数位对。

【输出格式】

输出到文件 `number.out` 中。

第一行一个正整数, 表示最大的 k 。

第二行一个长为 n 的 01 串, 表示 T 。第 i 个字符为 1/0 分别表示 $i \in T, i \notin T$ 。

【样例 1 输入】

```
1 3
2 2 2 3
3 1 3
4 0
```

【样例 1 输出】

```
1 2
2 111
```

【样例 1 解释】

$(1, 2), (1, 3), (2, 3)$ 是优美的数位对, 令 $T = \{1, 2, 3\}$ 中可以使每个数位和其它两个数位构成优美的数位对, $k_{max} = 2$ 。

【样例 2 输入】

```
1 6
2 2 2 4
3 1 3
4 1 4
5 1 5
6 1 6
7 0
```

【样例 2 输出】

```
1 2
2 111100
```

【样例 2 解释】

$(1, 2), (1, 4), (2, 3), (3, 4), (4, 5), (5, 6)$ 是优美的数位对。令 $T = \{1, 2, 3, 4\}$ 可以使每个数位都和 2 个其它 T 中的数位构成优美的数位对, $k_{max} = 2$ 。可以验证要使 $k = 2$, 5, 6 不能选入 T 中, 所以 $T = \{1, 2, 3, 4\}$ 是最大的。

【样例 3】

见选手目录下的 *number/number3.in* 与 *number/number3.ans*。
该样例满足测试点 1 ~ 6 的约束条件。

【样例 4】

见选手目录下的 *number/number4.in* 与 *number/number4.ans*。
该样例满足测试点 7 ~ 10 的约束条件。

【样例 5】

见选手目录下的 *number/number5.in* 与 *number/number5.ans*。
该样例满足测试点 11 ~ 13 的约束条件。

【样例 6】

见选手目录下的 *number/number6.in* 与 *number/number6.ans*。
该样例满足测试点 14 ~ 17 的约束条件。

【样例 7】

见选手目录下的 *number/number7.in* 与 *number/number7.ans*。
该样例满足测试点 18 ~ 20 的约束条件。

【数据范围】

设优美的数位对总数为 m 。保证 $m \leq \frac{n(n-1)}{2}$ 。

测试点编号	$n \leq$	$m \leq$	特殊限制
1 ~ 6	10	无	无
7 ~ 10	100	无	无
11 ~ 13	10^5	10^5	A
14 ~ 17	10^5	5×10^5	无
18 ~ 20	10^6	5×10^6	无

特殊性质 A：对任意一个数位，包含它的优美的数位对至多 2 对。

【提示】

输入量较大，建议使用较快的输入输出方式。可以以下面的模板为基础编写程序。
注意，如果你使用了以下代码，请只使用 `std::cin/std::cout` 进行读写，不要使用 `scanf/printf/getchar/putchar` 等 C 语言风格的读写方式。如果你是第一次使用较快的输入输出方式，请务必仔细测试文件读写是否正确。

```

1 #include <iostream>
2 int main() {
3     freopen("number.in", "r", stdin);
4     freopen("number.out", "w", stdout);
5     std::cin.tie(0)->sync_with_stdio(0);
6     // insert code here
7     return 0;
8 }
```

多边形 (polygon)

【题目描述】

小 R 喜欢玩小木棍。小 R 有 n 根小木棍，第 i ($1 \leq i \leq n$) 根小木棍的长度为 a_i 。

小 X 希望小 R 从这 n 根小木棍中选出若干根小木棍，将它们按任意顺序首尾相连拼成一个多边形。小 R 并不知道小木棍能拼成多边形的条件，于是小 X 直接将条件告诉了他：对于长度分别为 $l_1, l_2, \dots, l_m$ 的 m 根小木棍，这 m 根小木棍能拼成一个多边形当且仅当 $m \geq 3$ 且所有小木棍的长度之和大于所有小木棍的长度最大值的两倍，即 $\sum_{i=1}^m l_i > 2 \times \max_{i=1}^m l_i$ 。

由于小 R 知道了小木棍能拼成多边形的条件，小 X 提出了一个更难的问题：对每一个小木棍 i ，有多少种选择小木棍的方案，满足**这些小木棍的标号是一段连续的区间，这些小木棍包含了第 i 个小木棍**，使得选出的小木棍能够拼成一个多边形？你需要帮助小 R 求出。两种方案不同当且仅当选择的小木棍的**下标集合不同**，即存在 $1 \leq i \leq n$ ，使得其中一种方案选择了第 i 根小木棍，但另一种方案未选择。

为了减少输出量，你只需要输出 $\bigoplus_{i=1}^n (ans_i \times i)$ 。其中 ans_i 表示包含第 i 个小木棍的方案数。 $\oplus$ 表示二进制异或运算。

【输入格式】

从文件 *polygon.in* 中读入数据。

输入的第一行包含两个正整数 C, T ，表示测试点编号和测试数据数量。

每组测试数据有两行，第一行包含一个正整数 n ，表示小 R 的小木棍的数量。

第二行包含 n 个正整数 $a_1, a_2, \dots, a_n$ ，表示小 R 的小木棍的长度。

【输出格式】

输出到文件 *polygon.out* 中。

每组测试数据，输出一行一个非负整数，表示答案。

【样例 1 输入】

```

1 0 6
2 5
3 7 6 1 7 2
4 1
5 10000000000000
6 6
7 8 8 1 8 3 7

```



```
8 5
9 2 3 4 1 1
10 5
11 6 1 5 7 4
12 4
13 9 2 10 4
```

【样例 1 输出】

```
1 11
2 0
3 52
4 1
5 23
6 4
```

【样例 1 解释】

在第一组测试数据中，合法的区间有 $[1, 4]$, $[1, 5]$, $[2, 5]$ 。

$ans_1 = 2, ans_2 = 3, ans_3 = 3, ans_4 = 3, ans_5 = 2$ 。

答案为 $2 \oplus 6 \oplus 9 \oplus 12 \oplus 10 = 11$ 。

【样例 2】

见选手目录下的 *polygon/polygon2.in* 与 *polygon/polygon2.ans*。

该样例满足测试点 1 的约束条件。

【样例 3】

见选手目录下的 *polygon/polygon3.in* 与 *polygon/polygon3.ans*。

该样例满足测试点 5 的约束条件。

【样例 4】

见选手目录下的 *polygon/polygon4.in* 与 *polygon/polygon4.ans*。

该样例满足测试点 10 的约束条件。

【样例 5】

见选手目录下的 *polygon/polygon5.in* 与 *polygon/polygon5.ans*。
该样例满足测试点 15 的约束条件。

【样例 6】

见选手目录下的 *polygon/polygon6.in* 与 *polygon/polygon6.ans*。
该样例满足测试点 19 的约束条件。

【数据范围】

对于所有数据, $1 \leq T \leq 10^5$, $1 \leq n \leq 10^6$, $\sum n \leq 5 \times 10^6$, $1 \leq a_i \leq 10^{12}$ 。

测试点编号	$n \leq$	$\sum n \leq$	特殊性质
1 ~ 2	100	500	无
3 ~ 4	800	4000	无
5 ~ 6	5000	25000	无
7 ~ 9	10^5	5×10^5	无
10 ~ 12	3×10^5	1.5×10^6	无
13 ~ 14	6×10^5	3×10^6	有
15 ~ 18	6×10^5	3×10^6	无
19 ~ 20	10^6	5×10^6	有
21 ~ 25	10^6	5×10^6	无

特殊性质: a_i 在 $[1, 10^{12}]$ 内等概率随机。

【提示】

输入量较大, 建议使用较快的输入输出方式。可以以下面的模板为基础编写程序。
注意, 如果你使用了以下代码, 请只使用 `std::cin/std::cout` 进行读写, 不要使用 `scanf/printf/getchar/putchar` 等 C 语言风格的读写方式。如果你是第一次使用较快的输入输出方式, 请务必仔细测试文件读写是否正确。

```

1 #include <iostream>
2 int main() {
3     freopen("polygon.in", "r", stdin);
4     freopen("polygon.out", "w", stdout);
5     std::cin.tie(0)->sync_with_stdio(0);
6     // insert code here
7     return 0;

```

8 }

异或和 (xor)

【题目描述】

小 R 有一个边带权的简单无向连通图 G ，点集为 $1 \sim n$ ，有 m 条边。定义一个边子集 $E \subseteq G$ 是合法的当且仅当 E 内的边可以将 $1 \sim n$ 连通，即从 1 点开始，走若干条 E 中的边，可以走到所有点。定义一个边子集的权值为其边权异或和。请你帮他求出所有合法边子集的权值异或和。

【输入格式】

从文件 `xor.in` 中读入数据。

第一行两个正整数 n, m ，代表点数与边数。

接下来 m 行，每行三个非负整数 u, v, w 代表一条连接 u, v ，边权为 w 的边。

【输出格式】

输出到文件 `xor.out` 中。

一行一个非负整数，代表所有合法边子集的权值异或和。

【样例 1 输入】

```
1 3 3
2 1 2 1
3 2 3 2
4 1 3 4
```

【样例 1 输出】

```
1 7
```

【样例 1 解释】

合法的边子集有：

- $\{(1, 2), (2, 3)\}$ ，权值为 $1 \oplus 2 = 3$ ；
- $\{(2, 3), (1, 3)\}$ ，权值为 $2 \oplus 4 = 6$ ；
- $\{(1, 2), (1, 3)\}$ ，权值为 $1 \oplus 4 = 5$ ；
- $\{(1, 2), (2, 3), (1, 3)\}$ ，权值为 $1 \oplus 2 \oplus 4 = 7$ ；

所以所有合法边子集的权值异或和为 $3 \oplus 6 \oplus 5 \oplus 7 = 7$ 。

边集 $\{(1, 2)\}$ 不合法，因为 1 无法仅通过边集中的边走到 3。

【样例 2 输入】

```
1 4 6
2 1 2 1
3 2 3 1
4 3 4 1
5 1 3 1
6 2 4 1
7 1 4 1
```

【样例 2 输出】

```
1 0
```

【样例 3】

见选手目录下的 *xor/xor3.in* 与 *xor/xor3.ans*。
该样例满足子任务 1 的约束条件。

【样例 4】

见选手目录下的 *xor/xor4.in* 与 *xor/xor4.ans*。
该样例满足子任务 2 的约束条件。

【样例 5】

见选手目录下的 *xor/xor5.in* 与 *xor/xor5.ans*。
该样例满足子任务 7 的约束条件。

【样例 6】

见选手目录下的 *xor/xor6.in* 与 *xor/xor6.ans*。
该样例满足子任务 8 的约束条件。

【数据范围】

对于所有数据，保证：

- $2 \leq n \leq 10^6$;
- $1 \leq m \leq 10^6$;

- $1 \leq u, v \leq n$;
- $0 \leq w < 2^{31}$;
- G 中无重边与自环, 且 G 连通。

子任务编号	$n, m \leq$	特殊限制	分值
1	20	无	10
2	10^6	A	10
3	1000	B	5
4	1000	无	20
5	10^5	B	10
6	10^5	无	10
7	10^6	B	10
8	10^6	无	25

特殊性质 A: $m = n - 1$;

特殊性质 B: $m = n$ 。

【提示】

输入量较大, 建议使用较快的输入输出方式。可以以下面的模板为基础编写程序。注意, 如果你使用了以下代码, 请只使用 `std::cin/std::cout` 进行读写, 不要使用 `scanf/printf/getchar/putchar` 等 C 语言风格的读写方式。如果你是第一次使用较快的输入输出方式, 请务必仔细测试文件读写是否正确。

```

1 #include <iostream>
2 int main() {
3     freopen("xor.in", "r", stdin);
4     freopen("xor.out", "w", stdout);
5     std::cin.tie(0)->sync_with_stdio(0);
6     // insert code here
7     return 0;
8 }
```